

Nyakizu Digital Market: A Community-Based Digital Platform for Trusted Phone Accessories Trade

Final Project Completion Report

Nzabakamira Shema Manasseh

Software Engineering Project I (SWE3090XA)

Dr Ogore Fredric Michael

August 10, 2026

Declaration

I, Shema Nzabakamira Manassé, declare that this final report and the Nyakizu Digital Market platform it describes are my own work. The system documented here was designed, built, deployed, and tested by me over the course of this project, building directly on the proposal I submitted earlier in this course.

This report is written honestly: it covers what was actually built and shipped, not just what was originally planned. Where the finished system differs from the proposal, or where something planned was simplified or left for later, I say so directly rather than pretending the original plan was followed to the letter.

Acknowledgement

I again want to acknowledge the Banyamulenge phone accessory wholesalers, retailers, and mobile vendors in Nairobi whom I have worked with as both a wholesaler and a reseller. Their patience with my questions, and their honesty about what actually slows their businesses down day to day, is the reason this platform looks the way it does.

I also want to thank Dr Ogore Fredric Michael for the guidance given throughout this course, which pushed this project from a rough idea into something that could actually be deployed and used.

Dedication

This project remains dedicated to the hardworking business owners and traders in the Banyamulenge network. Watching them keep their part of the local economy running, and seeing them stay open to trying a new tool that could make their trading easier, is what kept this project moving from proposal to a working, deployed system.

Abstract

This report covers the completed build of the Nyakizu Digital Market, a web application that helps digitize how Banyamulenge phone accessories traders in Nairobi run their business. The proposal for this project set out to replace WhatsApp threads and paper notebooks with a platform that keeps the trust-based, relationship-first way these traders already work, while adding proper record-keeping.

The finished MVP is a mobile-first Progressive Web App with a Next.js and TypeScript frontend and a Django REST Framework backend, deployed and reachable online rather than only running locally. Buyers (resellers) can request access to a specific seller's store, browse a catalog that only ever shows “available” or “out of stock” instead of exact stock counts, and submit an order that locks the moment it is sent so it cannot be quietly edited afterward. Sellers (wholesalers) review submitted orders, physically pack them against a checklist, set the final price once sourcing is done, and record payments against a running, append-only status history that doubles as the credit ledger. An admin role approves new seller accounts and can flag orders that need a closer look.

Beyond the four original objectives, the final build adds a few things that were not in the original proposal: a seller-side sales-insights dashboard, a packing checklist that survives page refreshes and device switches, and an order-flagging tool for admin oversight. One planned item — SMS or WhatsApp order notifications — was deliberately left for a later phase in favor of shipping working email notifications first; that decision, and a few other honest gaps between plan and delivery, are covered in Chapter Eight.

Getting the platform actually live also surfaced problems no amount of local testing would have caught — a free-tier host blocking outbound SMTP, memory limits forcing a smaller worker count, and a reverse-proxy quirk that silently dropped trailing slashes on API calls. Chapter Seven walks through each of these and how they were fixed, because that debugging is as much a part of this project's result as the features themselves.

Acronyms and Definitions

API	Application Programming Interface
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
DRF	Django REST Framework
ERD	Entity Relationship Diagram
JWT	JSON Web Token
KES	Kenyan Shillings
MVP	Minimum Viable Product
PWA	Progressive Web App
RBAC	Role-Based Access Control
SMTP	Simple Mail Transfer Protocol
SRS	Software Requirements Specification
SSDLC	Secure Software Development Life Cycle

Table of Contents

Declaration	2
Acknowledgement	3
Dedication	4
Abstract	5
Acronyms and Definitions	6
Chapter One: Introduction	9
1.1 Recap of the Problem	9
1.2 What This Report Covers	9
1.3 Objectives Revisited	9
Chapter Two: Literature Review, Revisited	10
2.1 Why Existing Tools Still Don't Fit	10
2.2 What Guided the Final Design	10
Chapter Three: Final System Overview	10
3.1 Who Uses the Platform	10
3.2 How the Roles Differ From the Original Plan	11
Chapter Four: System Architecture, As Built	11
4.1 Technology Stack Actually Used	11
4.2 Context Diagram, As Built	11
4.3 Data Flow, As Built	12
4.4 Entity Relationship Diagram, As Built	12
Chapter Five: Features Delivered	13
5.1 Accounts, Roles, and Trust	13
5.2 Product Catalog	13
5.3 Ordering Workflow and Order Locking	13
5.4 Seller Packing Checklist	13
5.5 Credit Ledger and Debt Tracking	14
5.6 Admin Oversight and Order Flagging	14
5.7 Notifications	14

5.8 Offline Support and Installable App	14
5.9 Seller Sales Insights (Bonus Feature)	15
Chapter Six: Testing	15
6.1 Automated Tests	15
6.2 Manual and Device Testing	15
6.3 Production Smoke Testing	15
Chapter Seven: Deployment and Real-World Problems	16
7.1 Where It's Hosted	16
7.2 Problems Hit After Going Live, and How They Were Fixed	16
Chapter Eight: How the Final Build Differs From the Proposal	17
8.1 Admin Role Simplified	17
8.2 SMS / WhatsApp Notifications Deferred	17
8.3 Ledger Implemented as Balances Plus a Status Trail	17
8.4 "Draft" Is Client-Side, Not a Server Status	17
Chapter Nine: Results and Discussion	18
9.1 Objectives Checked Against the Finished Platform	18
9.2 What This Means	18
Chapter Ten: Conclusion and Future Work	18
10.1 Conclusion	18
10.2 Future Work	18
References	19

Chapter One: Introduction

1.1 Recap of the Problem

The proposal for this project laid out the situation clearly: Banyamulenge phone accessories traders in Nairobi run real, well-connected businesses almost entirely on personal trust, verbal agreements, WhatsApp messages, and paper notebooks. That works when a business is small, but it breaks down as product lists grow, as more money moves on credit, and as buyers and sellers both lose track of who owes what. Standard e-commerce platforms like Shopify or Jumia do not fit either, because they are built for open competition and show everyone exactly what is in stock — which is the opposite of how this community wants to operate.

This final report does not repeat that whole argument. It picks up where the proposal left off and reports on what was actually built to answer it, what worked, what changed along the way, and what is left for a future phase.

1.2 What This Report Covers

This report walks through the finished Nyakizu Digital Market platform as it stands today: a live, deployed web application with real user roles, a real database, and real email notifications going out. It covers the system's architecture and design as actually implemented, the features that were delivered, the testing that was done, the problems hit while deploying it for real, and an honest comparison between what was proposed and what shipped.

1.3 Objectives Revisited

The proposal set one main objective and four specific objectives. It is worth restating them here, because Chapter Nine measures the finished platform against each one directly:

- **Main objective:** Build a Progressive Web App that lets resellers put together digital order lists easily, while wholesalers keep full control over order status and their own financial records.
- **Objective 1:** A secure digital ordering system that locks an order once submitted, so no one can quietly change it while the wholesaler is fulfilling it.
- **Objective 2:** Flexible fulfillment through general availability labels instead of exact stock numbers, so sellers can freely source missing items to complete a big order.
- **Objective 3:** A permanent, add-only credit ledger controlled by the wholesaler, so financial records stay accurate and cannot be silently edited.
- **Objective 4:** Offline features that save order drafts locally and sync automatically once the connection returns.

Chapter Two: Literature Review, Revisited

2.1 Why Existing Tools Still Don't Fit

The literature review in the original proposal compared this project against two kinds of tools: basic chat apps like WhatsApp, which have no record-keeping built in at all, and full e-commerce platforms like Shopify, Jumia, and WooCommerce, which have record-keeping but force public transparency — exact stock counts, open pricing, no concept of a private, relationship-gated storefront. Building the actual platform confirmed that gap was real. None of the shortcuts considered along the way (using a generic storefront builder, or just adding a spreadsheet on top of WhatsApp) would have supported private, seller-approved buyer relationships or an order-locking workflow without a lot of awkward, brittle glue code.

2.2 What Guided the Final Design

Three ideas from the literature review carried through into the actual build unchanged: keep exact inventory numbers private, give the wholesaler full control over pricing and the ledger, and let the app degrade gracefully when the connection drops. All three show up directly in the code, described in Chapter Five — the product serializer never exposes a raw stock count to buyers, order totals can only be finalized by the seller, and the frontend keeps an IndexedDB store of drafts and a sync queue specifically so a spotty connection does not lose a buyer's order list.

Chapter Three: Final System Overview

3.1 Who Uses the Platform

The finished platform has three real user roles, each with its own part of the app:

- **Buyer (reseller):** Registers, verifies their email, requests access to a specific seller's store, browses that seller's catalog, builds and submits an order, tracks its status, and can see exactly what they still owe on the debts page.
- **Seller (wholesaler):** Registers a store profile that must be approved by an admin before it goes live, manages a product catalog with categories and variant attributes, reviews and sources submitted orders, sets the final price, records payments, and can see a ledger of every buyer's balance.
- **Admin:** Approves or rejects new seller accounts, and can flag or unflag any order that needs a closer look — a dispute, a suspected mistake, or a cancellation that should not go through quietly.

3.2 How the Roles Differ From the Original Plan

The proposal's context diagram described four separate actors: Reseller, Wholesaler, System Admin, and System Administrator, with the last two doing similar-sounding jobs — approving reseller accounts versus approving wholesaler accounts and handling platform security. In the actual build, those two admin-ish roles collapsed into one: a single **admin** role, tied directly to Django's built-in *staff* and *superuser* flags, handles both seller approval and order oversight. This was a deliberate simplification once building started — for an MVP with one operator running the platform, splitting admin duties into two separate roles added complexity without adding real value, and it can be split apart again later if the platform grows enough to need it.

Chapter Four: System Architecture, As Built

4.1 Technology Stack Actually Used

The decoupled frontend/backend split from the proposal held up through the whole build:

- **Frontend:** Next.js (App Router) with TypeScript and Tailwind CSS, built as an installable Progressive Web App using a Serwist-generated service worker for asset caching and an offline fallback page, plus a hand-written IndexedDB layer for order drafts and a background sync queue.
- **Backend:** Python with Django and the Django REST Framework, handling authentication, role-based permissions, order and ledger business logic, and the API the frontend talks to.
- **Database:** PostgreSQL in production, matching the proposal's choice for reliable, accurate transaction records.
- **Hosting:** The platform is deployed and reachable online rather than only demoed locally — both the frontend and backend run on Render, which shaped several of the real-world fixes described in Chapter Seven.
- **Email:** Order notifications are sent through Resend, an HTTPS email API, after an earlier attempt using direct Gmail SMTP ran into hosting-platform limits (also covered in Chapter Seven).

4.2 Context Diagram, As Built

The system still sits as the central hub connecting Buyers, Sellers, and Admins, matching the shape of the proposal's context diagram, just with the two admin actors merged into one as explained in section 3.2. Buyers send in procurement requests and payment references and read back their order status and balance; Sellers send in pricing updates, sourcing notes, and payment confirmations and read back submitted orders; Admins send in approval and flag decisions and read back the list of pending sellers and flagged orders.

4.3 Data Flow, As Built

The order lifecycle that was sketched in the proposal's data flow diagram is exactly what ended up in the *Order.STATUS_CHOICES* field in the code, with one addition. An order moves through: **submitted** -> **sourcing** -> **locked** -> **debt_active** (if a partial payment leaves a balance) -> **cleared**, with a **cancelled** state reachable from earlier stages. Every single transition is written to an append-only *OrderStatusEvent* row before anything else happens, which is what powers both the buyer-facing status tracker and the email notification sent on every change — described together in section 5.6.

4.4 Entity Relationship Diagram, As Built

The core tables and how they connect stayed close to the proposal's plan, with a few refinements that came out of actually building the ordering and catalog features:

- **CustomUser:** One user table with a role field (buyer, seller, admin) instead of separate user types, with Django's staff and superuser flags kept structurally in sync with the admin role so an admin account can never accidentally fail a permission check.
- **BuyerProfile / SellerProfile:** One-to-one profile tables holding role-specific detail — business type and location for buyers; store name, description, categories, and an approval workflow (pending, approved, rejected, needs more information) for sellers.
- **BuyerSellerRelationship:** The buyer-must-be-approved-by-seller trust gate from the proposal, implemented as its own table with a pending/approved/denied status, enforced in the order-creation serializer so an unapproved buyer cannot place an order at all.
- **Category / CategoryAttribute / AttributeValue:** The product catalog grew one level deeper than originally planned: instead of a flat subcategory list, each category can define its own set of variant axes (for example, a Screen Protectors category with “Phone Model”, “Material”, and “Function” as separate, independently selectable dimensions).
- **Product:** Belongs to one seller and one category, carries the actual *stock_quantity* the seller sees, but only ever exposes an available/out-of-stock status to buyers — the exact number never leaves the backend for a buyer-facing request.
- **Order / OrderItem / OrderStatusEvent:** An order belongs to one buyer and one seller and holds one or more items, each snapshotting its unit price at the time of purchase so a later price change cannot retroactively change what was already ordered. The append-only status event log doubles as the ledger's audit trail.

Chapter Five: Features Delivered

5.1 Accounts, Roles, and Trust

Registration collects a role (buyer or seller) up front and sends a verification email with a token that expires after twenty-four hours, so an unverified or fake account cannot act on the platform. A seller account additionally sits in a pending state until an admin reviews and approves the store, exactly as the proposal described. On the buyer side, before a reseller can place a single order with a given seller, they have to request access to that seller's store and be approved — this is the feature that most directly protects the trust-based culture the whole project is built around, since it means a seller only ever deals with buyers they have chosen to work with.

5.2 Product Catalog

Sellers list products under categories, optionally attaching variant attributes (phone model, material, colour, and so on) so a single product listing can represent several real-world variants without duplicating entries. Every product carries a status of available, out of stock, or draft, and buyers only ever see that status label, never the underlying stock count — delivering on the proposal's second specific objective directly.

5.3 Ordering Workflow and Order Locking

A buyer builds a list of items against one seller's catalog, adds optional notes and a delivery address, and submits it. The moment it is submitted, the order is locked from the buyer's side: they can track it, but they cannot edit the line items anymore. This satisfies the proposal's first specific objective and was one of the least-negotiable requirements throughout the build, because it is the one feature that most directly stops the “late order changes” problem the traders described.

5.4 Seller Packing Checklist

This is a feature that was not in the original proposal, added after watching how a seller actually fulfills an order in real life. While an order sits in the *sourcing* stage, the seller gets a checklist — one line per item — that they tick off as each item physically goes into the bag. The state is saved on the server per item, not just in the browser, so a seller can close the tab, switch devices, or come back an hour later without losing progress. It is a small operational tool, but it came directly out of thinking about the actual physical act of fulfilling an order, not just the data around it.

5.5 Credit Ledger and Debt Tracking

Rather than a separate ledger table with its own rows, the finished system tracks debt through each order's running balance (final price minus amount paid so far) combined with the append-only status event log described in section 4.3. A seller's ledger view lists every buyer's outstanding balance across all their orders; a buyer's debts page shows the same information from their side. Payments are recorded manually by the seller against a payment reference (an M-Pesa transaction code, for example) and a payment method, and every payment recalculates the order's balance and status automatically. This is a close but not identical match to the proposal's “permanent add-only credit ledger” objective — the ledger effect (an accurate, tamper-resistant record of who owes what) is delivered, but through order-level balances plus a status trail rather than a dedicated ledger-entry table. Chapter Eight discusses this trade-off honestly.

5.6 Admin Oversight and Order Flagging

An admin can pull up a list of all orders across the platform and flag any order that needs attention, with a reason attached, and unflag it once resolved. This is deliberately a lightweight tool — a flag and a reason field, not a full support-ticket system — built to give an admin a way to find and mark problem orders without the cost of building out a whole dispute-resolution workflow the platform did not yet need.

5.7 Notifications

Every order status change fires an email to the buyer, and a seller gets a separate “new order” email the moment an order is submitted to their store. This runs through one single function that every status-changing action in the codebase calls, which matters more than it sounds — it is what guarantees a buyer is notified consistently no matter which of the several places in the code changed their order's status, and it is what stops a routine database save from accidentally sending a duplicate notification. SMS or WhatsApp notifications, which the proposal's “emerging issues” section flagged as important given how much this community already lives on WhatsApp, were deliberately left out of this phase in favor of getting working, reliable email notifications shipped first. That decision is revisited in Chapter Ten.

5.8 Offline Support and Installable App

The frontend is a genuine installable Progressive Web App: a service worker caches the app shell and shows a dedicated offline page when there is no connection, and the browser offers an install prompt so a trader can add it to their home screen like a native app. On top of that, order-building specifically supports the proposal's fourth objective: while a buyer is offline, their in-progress order list is saved into a local IndexedDB store keyed to the seller they are ordering from, and if they submit while offline, the order goes into a sync queue that is retried once the connection comes back, rather than being lost.

5.9 Seller Sales Insights (Bonus Feature)

Also outside the original scope, the seller dashboard includes a sales-insights panel: a fourteen-day trend of order value placed per day, plus ranked lists of top products by units ordered and top buyers by order count. It is computed entirely from data the seller already has access to — their own order list — so it did not need a new backend endpoint, and it gives a seller a lightweight sense of how their business is trending without needing to export anything or do the counting by hand.

Chapter Six: Testing

6.1 Automated Tests

Each Django app in the backend — accounts, products, and orders — has its own test suite covering the behaviour that matters most for trust and correctness: that an unapproved buyer cannot place an order with a seller they have not been approved by, that an order cannot be edited by the buyer once submitted, that a seller's exact stock count never appears in a buyer-facing response, and that role permissions actually block the requests they are supposed to block. These tests also acted as a safety net during the project: at one point the buyer-seller approval feature's serializer and views had gone missing from the codebase during earlier refactoring, and the existing test suite is what caught it and made the regression obvious enough to fix quickly.

6.2 Manual and Device Testing

Following the proposal's plan, the app was tested on older, low-spec Android phones rather than only on a development machine, since that is the hardware the traders themselves actually use. Testing on that hardware kept pressure on keeping pages light and interactions simple, and surfaced usability issues — like contrast and copy that assumed more English fluency than the audience has — that would not have shown up on a fast desktop browser.

6.3 Production Smoke Testing

Beyond local and device testing, the platform was tested against its real, deployed production environment: real verification and order-status emails were sent and confirmed received during development, and the login, checkout, and fulfillment flows were walked through end-to-end against the live Render deployment after each infrastructure change described in Chapter Seven, to confirm the fix actually worked in the environment traders would use, not just in theory.

Chapter Seven: Deployment and Real-World Problems

7.1 Where It's Hosted

Both the frontend and the backend are deployed on Render, with the Next.js frontend proxying API calls through to the Django backend. Choosing to actually deploy the platform, rather than just demoing it locally, was a deliberate decision: it surfaced a set of real production problems that never would have shown up in local development, and fixing them is arguably as much a part of this project's engineering work as building the features themselves.

7.2 Problems Hit After Going Live, and How They Were Fixed

A handful of issues only appeared once the platform was actually running on Render, each requiring a real fix rather than a workaround:

- **Outbound email silently failing:** Render's free tier blocks outbound SMTP ports, so the original plan of sending verification and order emails directly through Gmail's SMTP server did not work in production even though it worked locally. This was fixed in two steps: first forcing IPv4 for outbound SMTP to rule out a DNS/IPv6 delivery issue, and then, once the port-blocking cause was confirmed, switching email sending over to Resend, an HTTPS-based email API that does not depend on an SMTP port at all.
- **Memory limits on the free tier:** The backend's gunicorn worker count had to be reduced to fit inside the 512MB RAM ceiling of Render's free tier, trading a little request concurrency for a backend that does not get killed for running out of memory.
- **Silent redirect loop on API calls:** The frontend's proxy rewrite for `/api/` requests was dropping trailing slashes, which caused Django's own slash-redirect behaviour to loop forever on some requests instead of reaching the backend. This was traced and fixed at the proxy layer.
- **Cross-site cookies getting blocked:** Because the frontend and backend are on different subdomains, browsers were blocking the authentication cookie as a cross-site cookie by default. The fix was to route API calls through the frontend's own origin instead of calling the backend directly, so the cookie is always same-site from the browser's point of view.
- **Leaving debug tooling on in production:** Django REST Framework's browsable API renderer — handy for local development, but a data-exposure risk if left on publicly — was explicitly disabled in the production configuration.

None of these problems were visible while developing locally. They only appeared once real traffic hit a real, resource-constrained, publicly reachable host, which is exactly why they are worth recording here rather than treating deployment as an afterthought to the actual software.

Chapter Eight: How the Final Build Differs From the Proposal

Part of reporting on a finished project honestly is being clear about where the plan and the delivery do not perfectly match. None of the differences below are hidden shortcomings — each was a deliberate call made once real constraints or real usage patterns became clear during the build.

8.1 Admin Role Simplified

The proposal's context diagram split administration into two actors (System Admin approving resellers, System Administrator handling wholesaler verification and platform security). The finished platform merges these into a single admin role. For a single-operator MVP this removed unnecessary complexity; splitting the role back apart is a small, well-contained change if the platform grows enough to need separate moderation and technical-admin staff.

8.2 SMS / WhatsApp Notifications Deferred

The proposal's emerging-issues section pointed out that this community already lives on WhatsApp, which made SMS or WhatsApp order notifications an obvious thing to want. That integration (through a provider such as Africa's Talking or Twilio) was deliberately not built in this phase, in favor of shipping working, tested email notifications first. No plumbing for SMS/WhatsApp exists yet in the codebase; it is the clearest, highest-value item for a next phase, covered again in Chapter Ten.

8.3 Ledger Implemented as Balances Plus a Status Trail

The proposal described a “permanent, add-only credit ledger” as its own thing. What actually got built achieves the same outcome — an accurate, tamper-resistant record of what a buyer owes — through each order's running balance combined with the append-only *OrderStatusEvent* log, rather than through a separate ledger-entry table with one row per transaction. Functionally this covers the same ground for the MVP's needs, but a dedicated ledger table with individual payment-line entries (rather than a single running *amount_paid* per order) would give a cleaner audit trail if the platform needs to support partial-payment history in more detail later.

8.4 “Draft” Is Client-Side, Not a Server Status

The proposal's order lifecycle diagram listed *Draft* as a server-tracked stage before *Submitted*. In the final build, a draft order lives entirely on the buyer's device (in the IndexedDB store described in section 5.8) until the moment it is actually submitted, at which point it becomes a real *Order* row on the backend for the first time. This still delivers the intended behaviour — a buyer's in-progress list is never lost, and a seller never sees a half-finished order — just through a different mechanism than originally sketched.

Chapter Nine: Results and Discussion

9.1 Objectives Checked Against the Finished Platform

Measuring the finished system against the four specific objectives from section 1.3:

- **Order locking (Objective 1):** Delivered. An order cannot be edited by the buyer after submission.
- **Availability labels, not stock counts (Objective 2):** Delivered. Buyers only ever see available/out of stock; the exact quantity stays backend-only.
- **Wholesaler-controlled ledger (Objective 3):** Delivered, with the implementation difference described in section 8.3.
- **Offline drafts with auto-sync (Objective 4):** Delivered. Drafts are saved locally and queued orders sync automatically once the connection returns.

9.2 What This Means

All four specific objectives were met, and the main objective — a PWA that gives resellers an easy digital ordering experience while keeping wholesalers in control of status and finances — holds up in the finished platform. The two feature areas that go beyond the original scope (the packing checklist and the sales-insights dashboard) came directly out of thinking through how a seller actually works day to day, not from feature creep for its own sake, which is the same trader-first approach the original proposal argued for.

Chapter Ten: Conclusion and Future Work

10.1 Conclusion

This project set out to build a digital platform that respects how Banyamulenge phone accessories traders already do business, instead of asking them to change how they work to fit a generic e-commerce tool. The finished, deployed Nyakizu Digital Market delivers on that: it locks orders to stop last-minute disputes, keeps stock numbers private to protect sourcing flexibility, gives wholesalers full control over pricing and payment records, and keeps working when the connection drops. Getting it actually live on a real host, rather than stopping at a local demo, forced the project to deal with problems — blocked SMTP ports, memory limits, proxy quirks, cross-site cookies — that a purely local build would never have had to face, and solving those is a real part of what this project delivered.

10.2 Future Work

- **SMS and WhatsApp notifications:** The highest-value next step, given how much this community already relies on WhatsApp — needs a provider decision (Africa's Talking versus Twilio versus another option) before building it.

- **Direct M-Pesa integration:** Payments are currently recorded manually against a reference code; a direct STK Push or callback-based integration would remove that manual step entirely.
- **A dedicated ledger-entry table:** As discussed in section 8.3, splitting payments into individual ledger rows rather than a single running balance would give a more detailed audit trail as transaction volume grows.
- **Splitting the admin role:** If the platform grows past one operator, separating seller-approval duties from platform/order oversight duties (as originally sketched in the proposal) becomes worth the added complexity.
- **Wider local-language support:** The platform already includes some Kinyarwanda placeholder copy for buyer-facing fields; a full pass at multi-language support would make the app more accessible to less English-fluent traders.

References

- Field observation and community-informed estimates. (2026). *Nyakizu digital market: Concept development notes* [Unpublished personal field notes]. Nairobi, Kenya.
- Nzabakamira, S. M. (2026). *Nyakizu Digital Market: A Community-Based Digital Platform for Trusted Phone Accessories Trade* [Project proposal, SWE3090XA]. United States International University–Africa.
- United Nations High Commissioner for Refugees. (2026a). *Kenya: Refugees and asylum-seekers in Kenya as of April 30, 2026*. UNHCR Operational Data Portal. <https://data.unhcr.org/en/country/ken>
- United Nations High Commissioner for Refugees. (2026b). *Kenya: Refugees and asylum-seekers in Nairobi as of April 30, 2026*. UNHCR Operational Data Portal. <https://data.unhcr.org/en/country/ken/187>